CS202 Design Pattern Report: Mediator Pattern

Centralized Airspace Traffic Coordination, $O(N^2)$ Mesh Decoupling, and Modern C++17 Architecture

Group 52
Nguyễn Đình Minh Huy (Student ID: 25125083)
Trần Gia Huy (Student ID: 25125084)

August 27, 2026

Contents

1 Real-world Problem & Motivation

1.1 Background: Airport Airspace and Runway Coordination

In modern aviation and distributed software systems, coordinating multiple independent, concurrent entities that contend for shared, safety-critical resources is a central architectural challenge. Consider an international airport managing active airspace within a terminal radar service area. Multiple types of aircraft operate simultaneously:

- **Commercial Passenger Jets (e.g., VN123, BA789):** Carrying hundreds of passengers on fixed flight schedules.
- **Heavy Cargo Transports (e.g., FX902):** Operating freight logistics with specific runway weight allowances.
- **Emergency Air Ambulances / Medevac (e.g., MEDIC1):** Transporting critical patients with low fuel reserves, requiring immediate priority runway clearance.

All aircraft must share a finite, exclusive physical resource: the active runway (`Runway 07L`). At most one aircraft may safely occupy the runway at any given instant for landing or takeoff.

1.2 The Communication Bottleneck: $O(N^2)$ Peer-to-Peer Mesh

If aircraft coordinate landing authorization directly with one another without a central coordinator (a peer-to-peer mesh architecture):

1. **Complete Graph Complexity:** For N active aircraft in the airspace, every aircraft must maintain direct radio and data links to all $N-1$ other aircraft. The total number of communication channels in the network forms a complete graph K_N with:
$$E = \frac{N(N-1)}{2} = O(N^2) \tag{1}$$
2. **High Entry/Exit Overhead:** When a new flight enters the airspace, it must establish N new bidirectional links. When a flight touches down and exits the airspace, all remaining $N-1$ aircraft must execute cleanup routines.
3. **Decentralized State Inconsistency and Deadlocks:** Because coordination logic is distributed across independent nodes, concurrent landing requests create race conditions, priority inversion, and potential mid-air collision hazards.

1.3 Problem Statement

The objective is to design a software architecture that enables aircraft to request landing authorization, handle holding patterns, receive runway vacancy cascades, and yield to emergency priority flights without aircraft having any compile-time or runtime knowledge of other individual aircraft instances.

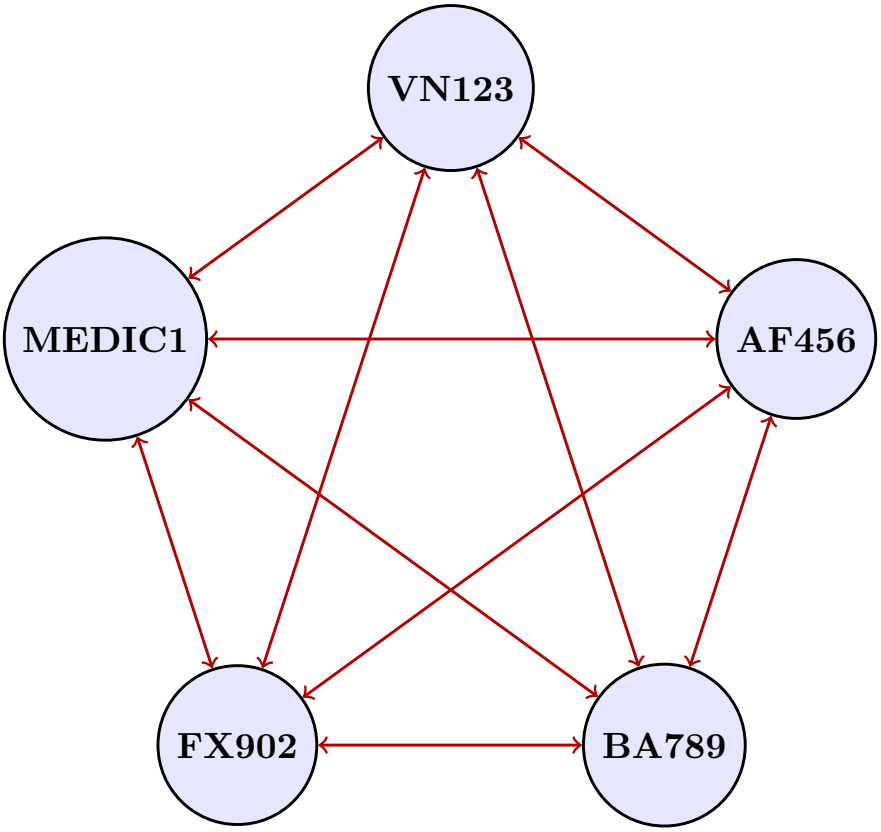

Figure 1: Decentralized Peer-to-Peer Flight Mesh with $O(N^2)$ Inter-Dependencies

2 Naive Solution

In a naive object-oriented design without a mediator, aircraft maintain explicit pointer collections of all peer aircraft instances. Each aircraft executes a decentralized consensus loop before attempting to acquire the runway.

```
namespace naive {

enum class FlightStatus {
    IN_FLIGHT,
    HOLDING_PATTERN,
    ON_RUNWAY,
    LANDED,
    PARKED_AT_GATE
};

enum class FlightType {
    COMMERCIAL,
    CARGO,
    EMERGENCY
};

class NaiveFlight {
private:
    std::string flightCode_;
    FlightType type_;
    FlightStatus status_;
    int capacityOrWeight_;
```

```
23
24      // Tightly coupled: Every flight maintains direct pointers to all active peers (O(N^2) mesh)
25      std::vector<NaiveFlight*> peerNetwork_;
26
27  public:
28      NaiveFlight(std::string code, FlightType type, int capacityOrWeight)
29          : flightCode_(std::move(code)),
30            type_(type),
31            status_(FlightStatus::IN_FLIGHT),
32            capacityOrWeight_(capacityOrWeight) {}
33
34      ~NaiveFlight() {
35          leaveAirspaceNetwork();
36      }
37
38      void addPeer(NaiveFlight* peer) {
39          if (peer != this && std::find(peerNetwork_.begin(), peerNetwork_.end(), peer) == peerNetwork_.end()) {
40              peerNetwork_.push_back(peer);
41          }
42      }
43
44      void removePeer(NaiveFlight* peer) {
45          peerNetwork_.erase(
46              std::remove(peerNetwork_.begin(), peerNetwork_.end(), peer),
47              peerNetwork_.end()
48          );
49      }
50
51      // Joins airspace: Wires this flight to all current flights, and all current flights to this flight
52      void joinAirspaceNetwork(std::vector<NaiveFlight*>& currentAirspace) {
53          for (auto* peer : currentAirspace) {
54              this->addPeer(peer);
55              peer->addPeer(this); // Bidirectional mutual registration
56          }
57          currentAirspace.push_back(this);
58      }
59
60      // Leaves airspace: Must notify and unhook from every peer
61      void leaveAirspaceNetwork() {
62          if (peerNetwork_.empty()) return;
63          for (auto* peer : peerNetwork_) {
64              peer->removePeer(this); // Mutate peer's internal collection
65          }
66          peerNetwork_.clear();
67      }
68
69      // Decentralized consensus algorithm: The flight polls all peers to determine if it is safe to land
70      bool requestLandingPermission() {
71          std::cout << "[" << flightCode_ << "] Requesting landing permission by polling "
```

```
                  << peerNetwork_.size() << " peers...\n";

        for (auto* peer : peerNetwork_) {
            // Check 1: Is any peer currently occupying the runway?
            if (peer->getStatus() == FlightStatus::ON_RUNWAY) {
                status_ = FlightStatus::HOLDING_PATTERN;
                std::cout << "  -> CONFLICT: Peer " << peer->getCode()
                          << " is currently ON_RUNWAY. Entering
    HOLDING_PATTERN.\n";
                return false;
            }

            // Check 2: Does an emergency peer have priority?
            if (type_ != FlightType::EMERGENCY && peer->getType() ==
    FlightType::EMERGENCY &&
                (peer->getStatus() == FlightStatus::HOLDING_PATTERN ||
    peer->getStatus() == FlightStatus::IN_FLIGHT)) {
                status_ = FlightStatus::HOLDING_PATTERN;
                std::cout << "  -> PRIORITY YIELD: Emergency peer " <<
    peer->getCode()
                          << " has priority. Entering HOLDING_PATTERN.\
    n";
                return false;
            }
        }

        // Consensus reached
        status_ = FlightStatus::ON_RUNWAY;
        std::cout << "  -> CONSENSUS APPROVED: All peers report runway
    clear. Status: ON_RUNWAY.\n";
        return true;
    }

    void touchdown() {
        if (status_ != FlightStatus::ON_RUNWAY) return;
        status_ = FlightStatus::LANDED;
    }

    void parkAtGateAndExitAirspace(std::vector<NaiveFlight*>&
    airspaceRegistry) {
        status_ = FlightStatus::PARKED_AT_GATE;
        airspaceRegistry.erase(
            std::remove(airspaceRegistry.begin(), airspaceRegistry.end
    (), this),
            airspaceRegistry.end()
        );
        leaveAirspaceNetwork();
    }
};

} // namespace naive
```

Listing 1: Naive Flight Coordination with Direct Peer Mesh (`naive.cpp`)

2.1 Step-by-Step Execution of the Naive Solution

Consider 3 aircraft (`VN123`, `AF456`, `BA789`) entering the airspace and contending for landing:

1. **Manual N-Way Mutual Connection:** When `VN123`, `AF456`, and `BA789` enter the airspace, each flight connects to all others, creating $3 \times 2 = 6$ direct pointer references.

2. **Decentralized Consensus Request:** Flight `VN123` calls the method `requestLandingPermission()`, querying `AF456` and `BA789`. It finds both in `IN_FLIGHT`, thereby acquiring `ON_RUNWAY`.

3. **Conflict and Rejection:** While `VN123` is on approach, `AF456` attempts to land. Its polling loop detects `VN123->status == ON_RUNWAY` and is forced into `HOLDING_PATTERN`.

4. **Ripple Cleanup on Exit:** When `VN123` touches down and parks at the terminal gate, it must execute an explicit $O(N)$ sweep across all peer vectors to remove its pointer.

3 Analysis of Naive Solution Drawbacks

Even when implemented using standard object-oriented idioms (proper enums, RAII destructors, encapsulation), the decentralized peer-to-peer approach suffers from fundamental architectural flaws:

Deficiency Area	Manifestation in Naive Code	Architectural Consequence
Single Responsibility Principle (SRP) Violation	`NaiveFlight` manages aerodynamic flight state while also coordinating distributed consensus, mutex locking, and peer lists.	Flight classes become bloated, mixing flight domain logic with network synchronization.
Open-Closed Principle (OCP) Violation	Adding new coordination rules (e.g., wake turbulence separation, weather delays) requires modifying the consensus loop in every flight class.	Existing flight classes must be altered and retested whenever airport coordination rules change.
Quadratic Link Complexity ($O(N^2)$)	Every flight stores direct pointers to all active peers in `peerNetwork_`.	Network link count scales quadratically with $N(N-1)$, causing memory and message explosion.
State Synchronization and Race Conditions	Flight states are polled across individual peers without centralized atomic locking.	Concurrent landing attempts can lead to simultaneous runway acquisition or deadlocks.
Dangling Pointer Hazards	Flights must execute an explicit $O(N)$ sweep to remove themselves from peers upon landing.	A single missed removal leaves a dangling pointer, causing fatal segmentation faults on the next cycle.

Table 1: Architectural and Performance Limitations of the Naive Flight Coordination Design

4 Introduction to the Mediator Design Pattern

4.1 Definition and Intent

According to the Gang of Four (GoF), the **Mediator Pattern** is defined as:

> *"Define an object that encapsulates how a set of objects interact. Mediator promotes loose coupling by keeping objects from referring to each other explicitly, and it lets you vary their interaction independently."*

The pattern replaces a chaotic, many-to-many communication mesh with a clean, one-to-many **Hub-and-Spoke (Star Topology)**.

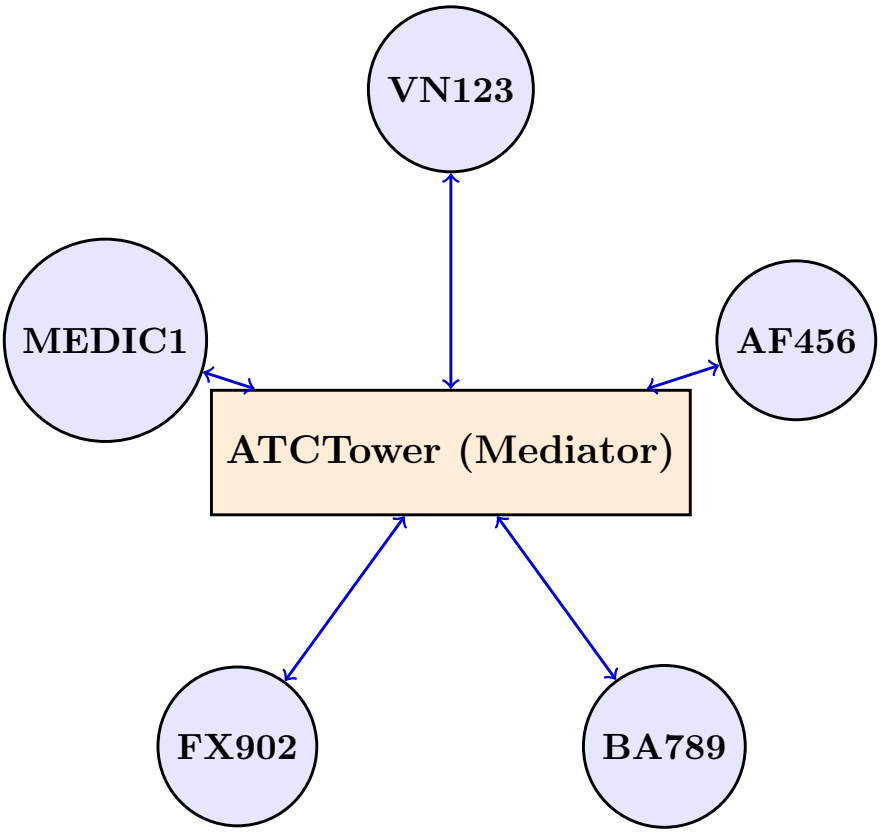

Figure 2: Mediator Star Topology ($O(N)$ Links): Colleagues Communicate Solely Through the Central Hub

4.2 Key Structural Participants

- **Mediator Interface (`IATCMediator`):** Declares the protocol for communication with colleague objects (e.g., `requestLanding()`, `notifyRunwayClear()`, `broadcastEmergency()`).
- **Concrete Mediator (`ATCTower`):** Implements cooperative behavior by coordinating colleagues. It maintains the single source of truth for runway state, landing queues, and priority overrides.
- **Colleague Base Class (`Flight`):** Defines the abstract colleague entity. Each colleague maintains a reference to its mediator object rather than referencing peer colleagues.
- **Concrete Colleagues (`CommercialFlight`, `CargoFlight`, `EmergencyFlight`):** Implement specialized domain behavior. When an event occurs, colleagues notify their mediator instead of contacting other colleagues.

5 UML Class Diagrams

5.1 General GoF Mediator Pattern Structure

The generic Mediator structure decouples colleague objects from one another by funneling all interaction logic through an abstract mediator interface.

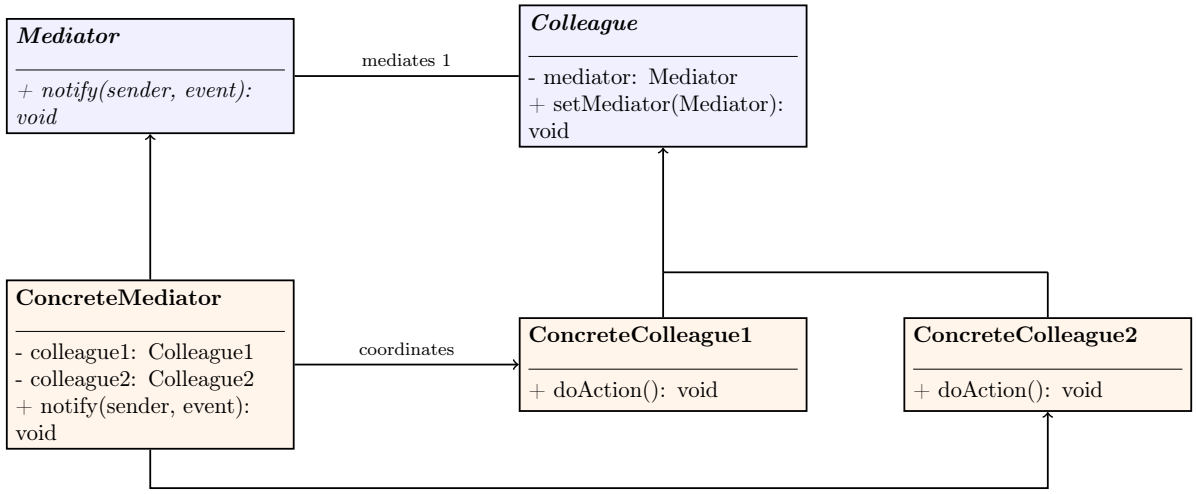

Figure 3: General GoF Mediator Pattern UML Structure

5.2 Airport Air Traffic Control Mediator Class Hierarchy

For our ATC runway coordination engine, `ATCTower` serves as the central mediator managing flight registrations, holding queues, and landing clearances across specialized flight colleagues.

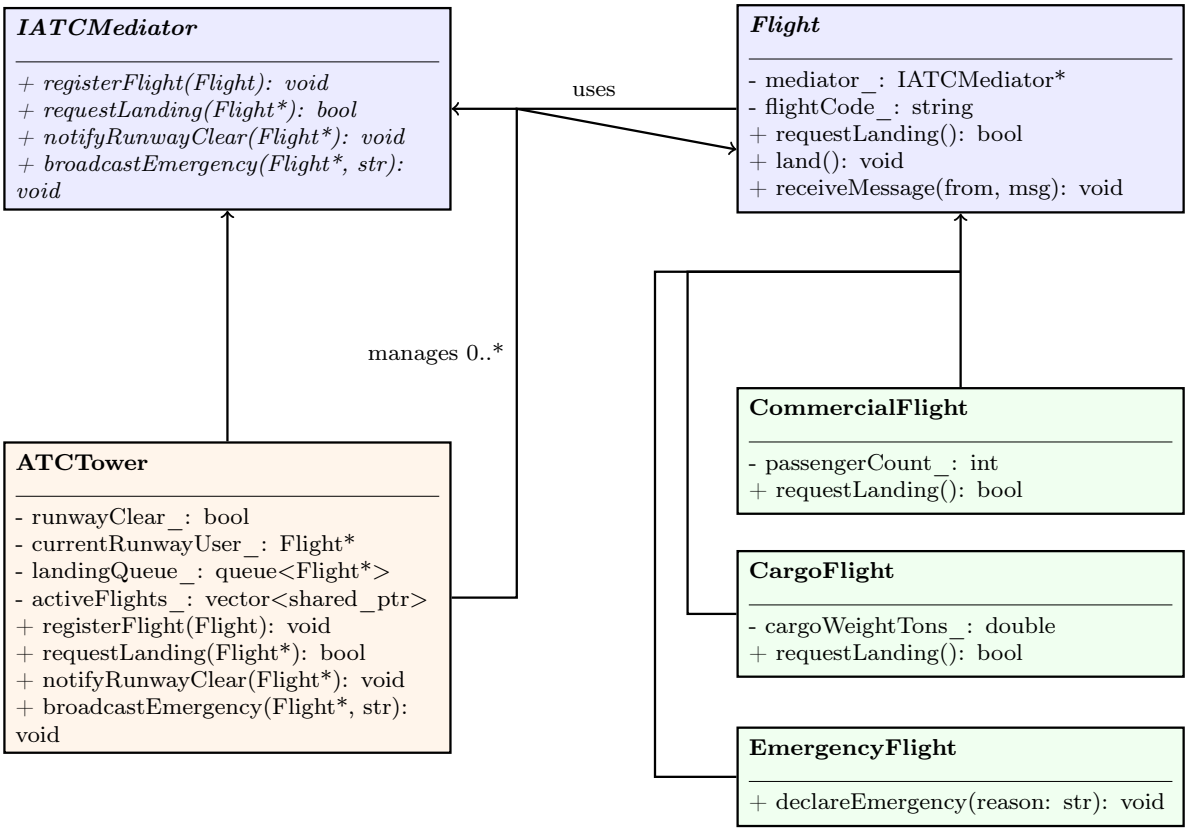

Figure 4: Airport Air Traffic Control Mediator Class Hierarchy

6 Pattern-Based Implementation in C++17

6.1 Architectural Design Rationale: Memory Management & Queuing

In our C++17 implementation:

- **Ownership Topology:** `ATCTower` maintains ownership of active airspace participants using `std::vector<std::shared_ptr<Flight>>`. Colleague instances store a non-owning raw pointer (`IATCMediator*`) to the mediator to prevent reference cycles.
- **FIFO Holding Pattern Queue:** When the runway is occupied, arriving flights are pushed to a `std::queue<Flight*> landingQueue_`.
- **Automatic Cascade on Vacation:** When the current runway user invokes `notifyRunwayClear()`, the tower automatically pops the next flight from the queue, grants clearance, and triggers its landing sequence.
- **Emergency Priority Overrides:** When an `EmergencyFlight` calls its `declareEmergency()` method, standard traffic is alerted. The emergency flight is then granted immediate runway access.

6.2 Source Code Implementation

The following listings demonstrate the complete, warning-free implementation (`Mediator/src/pattern.cpp`).

```
namespace pattern {

class Flight; // Forward declaration

// Abstract Mediator Interface
class IATCMediator {
public:
    virtual ~IATCMediator() = default;
    virtual void registerFlight(std::shared_ptr<Flight> flight) = 0;
    virtual bool requestLanding(Flight* flight) = 0;
    virtual void notifyRunwayClear(Flight* flight) = 0;
    virtual void broadcastEmergency(Flight* flight, const std::string&
   reason) = 0;
};

// Colleague Base Class
class Flight {
protected:
    IATCMediator* mediator_;
    std::string flightCode_;

public:
    Flight(std::string code, IATCMediator* mediator)
        : mediator_(mediator), flightCode_(std::move(code)) {}
    virtual ~Flight() = default;

    const std::string& getCode() const { return flightCode_; }

    virtual bool requestLanding() {
        std::cout << "[" << flightCode_ << "] Requesting landing
   clearance from ATC Tower...\n";
        return mediator_->requestLanding(this);
    }

    virtual void land() {
        std::cout << "[" << flightCode_ << "] >>> TOUCHDOWN SUCCESSFUL
   on Runway 07L. Clearing runway...\n";
        mediator_->notifyRunwayClear(this);
    }

    virtual void receiveMessage(const std::string& from, const std::
   string& msg) {
        std::cout << "  [" << flightCode_ << " Radio] Received from "
   << from << ": " << msg << "\n";
    }
};
```

Listing 2: Mediator Interface and Flight Colleague Base Class

```cpp
// Concrete Colleague: Commercial Passenger Flight
class CommercialFlight : public Flight {
private:
    int passengerCount_;

public:
    CommercialFlight(std::string code, int passengers, IATCMediator*
    mediator)
        : Flight(std::move(code), mediator), passengerCount_(passengers
    ) {}

    bool requestLanding() override {
        std::cout << "[" << flightCode_ << " (Commercial - " <<
    passengerCount_ << " passengers)] "
                  << "Approaching airport. Requesting landing clearance
    ...\n";
        return mediator_->requestLanding(this);
    }
};

// Concrete Colleague: Cargo Freight Flight
class CargoFlight : public Flight {
private:
    double cargoWeightTons_;

public:
    CargoFlight(std::string code, double cargoTons, IATCMediator*
    mediator)
        : Flight(std::move(code), mediator), cargoWeightTons_(cargoTons
    ) {}

    bool requestLanding() override {
        std::cout << "[" << flightCode_ << " (Cargo - " <<
    cargoWeightTons_ << " tons)] "
                  << "Heavy transport on final approach. Requesting
    landing clearance...\n";
        return mediator_->requestLanding(this);
    }
};

// Concrete Colleague: Emergency Flight (Air Ambulance / Medevac)
class EmergencyFlight : public Flight {
public:
    using Flight::Flight;

    void declareEmergency(const std::string& reason) {
        std::cout << "\n[" << flightCode_ << " (EMERGENCY)] !!! MAYDAY
    MAYDAY MAYDAY: " << reason << " !!!\n";
        mediator_->broadcastEmergency(this, reason);
    }
};
```

Listing 3: Concrete Colleagues: Commercial, Cargo, and Emergency Flights

```cpp
// Concrete Mediator: Air Traffic Control Tower
class ATCTower : public IATCMediator {
private:
    bool runwayClear_ = true;
    Flight* currentRunwayUser_ = nullptr;
    std::queue<Flight*> landingQueue_;
    std::vector<std::shared_ptr<Flight>> activeAirspaceFlights_;

public:
    void registerFlight(std::shared_ptr<Flight> flight) override {
        activeAirspaceFlights_.push_back(flight);
        std::cout << "[ATC Tower] Radar tracked: Flight " << flight->getCode()
                  << " entered airspace and registered with tower.\n";
    }

    bool requestLanding(Flight* flight) override {
        if (runwayClear_) {
            runwayClear_ = false;
            currentRunwayUser_ = flight;
            std::cout << "[ATC Tower] Clearance GRANTED for " << flight->getCode()
                      << ". Runway 07L is CLEAR for landing.\n";
            return true;
        } else {
            landingQueue_.push(flight);
            std::cout << "[ATC Tower] HOLDING PATTERN for " << flight->getCode()
                      << ". Runway 07L currently OCCUPIED by " << currentRunwayUser_->getCode()
                      << ". Added to queue (Queue position: " << landingQueue_.size() << ").\n";
            return false;
        }
    }

    void notifyRunwayClear(Flight* flight) override {
        if (currentRunwayUser_ == flight) {
            std::cout << "[ATC Tower] Runway 07L VACATED by " << flight->getCode() << ".\n";
            currentRunwayUser_ = nullptr;
            runwayClear_ = true;

            // Automatically dispatch the next flight waiting in queue
            if (!landingQueue_.empty()) {
                Flight* nextFlight = landingQueue_.front();
                landingQueue_.pop();
                std::cout << "[ATC Tower] Calling next queued flight: Clearance GRANTED for "
                          << nextFlight->getCode() << ".\n";
                runwayClear_ = false;
                currentRunwayUser_ = nextFlight;
                nextFlight->land();
            } else {
                std::cout << "[ATC Tower] Runway 07L is currently IDLE and ready for traffic.\n";
            }
        }
```

```
51      }
52
53      void broadcastEmergency(Flight* emergencyFlight, const std::string&
        reason) override {
54          std::cout << "[ATC Tower ALERT] Priority Emergency declared by
        " << emergencyFlight->getCode()
55                    << " (" << reason << "). Cleared for IMMEDIATE
        priority landing!\n";
56
57          // Broadcast safety alert to all other flights in airspace
58          for (const auto& f : activeAirspaceFlights_) {
59              if (f.get() != emergencyFlight) {
60                  f->receiveMessage("ATC Tower", "AIRSPACE ADVISORY:
        Yield runway for emergency flight "
61                                    + emergencyFlight->getCode() + " (" +
        reason + ").");
62              }
63          }
64
65          // Fast-track emergency landing
66          runwayClear_ = false;
67          currentRunwayUser_ = emergencyFlight;
68          emergencyFlight->land();
69      }
70  };
71
72  } // namespace pattern
```

Listing 4: Concrete Mediator: Central ATCTower Engine

6.3 Execution Walkthrough & Step-by-Step Simulation Trace

Table 2 summarizes the lifecycle events, runway states, and automatic queue dispatches during simulation:

Step	Event / Trigger	Runway	Holding Queue	ATC Tower Action / Dispatched Output
1	Radar Registration	CLEAR	Empty	`VN123`, `FX902`, `BA789` registered with central radar.
2	`VN123` Landing Req	OCCUPIED	Empty	Runway 07L is free; clearance granted to `VN123`.
3	`FX902` Landing Req	OCCUPIED	[`FX902`]	Runway occupied by `VN123`; `FX902` pushed to queue position 1.
4	`BA789` Landing Req	OCCUPIED	[`FX902`, `BA789`]	Runway occupied; `BA789` pushed to queue position 2.
5	`VN123` Vacates	OCCUPIED	[`BA789`]	Tower detects vacancy, pops `FX902`, and clears `FX902` for landing.
6	`FX902` Vacates	OCCUPIED	Empty	Tower detects vacancy, pops `BA789`, and clears `BA789` for landing.
7	`BA789` Vacates	CLEAR	Empty	`BA789` vacates runway; runway status transitions to IDLE.
8	`MEDIC1` Mayday	OCCUPIED	Standard Halted	Tower broadcasts advisory to all aircraft; `MEDIC1` lands with immediate priority.

Table 2: Dynamic Execution and Queue Cascade Trace in the Mediator System

7 Evaluation of Trade-offs

7.1 Advantages

- **Decoupled Colleagues ($O(N)$ vs. $O(N^2)$ Complexity):** Colleagues no longer reference peers. Flights communicate exclusively with `IATCMediator`, drastically reducing class inter-dependencies.
- **Centralized Control & Business Logic (SRP Compliance):** Runway assignment policies, separation standards, and holding pattern queues are centralized in `ATCTower`, freeing colleague classes to focus solely on flight mechanics.
- **High Extensibility (OCP Compliance):** New flight categories (e.g., `Helicopter`, `AutonomousDroneDelivery`) can be introduced without modifying existing flight implementations.
- **Elimination of Dangling Pointers:** When an aircraft lands and exits the airspace, it unregisters from the central mediator in $O(1)$ time with zero ripple effects across other flight objects.

7.2 Disadvantages and Limitations

- **"God Object" Anti-Pattern Risk:** As a system evolves, business rules, filtering policies, and coordination protocols can cause the ConcreteMediator to become monolithic, overly complex, and difficult to maintain.
- **Single Point of Failure / Bottleneck:** If the mediator crashes or experiences thread lockups, communication between all colleagues ceases entirely.
- **Indirection Overhead:** Direct method calls between colleagues are replaced with virtual function dispatch through the mediator, adding a minor layer of runtime indirection.

8 Applications

The Mediator pattern is widely used in modern software engineering to manage complex interaction flows:

- **ASP.NET Core MediatR Library:** In enterprise CQRS (Command Query Responsibility Segregation) web applications, MediatR acts as an in-process mediator. Web API controllers send Command or Query objects to MediatR without knowing which handler processes the request. Cross-cutting pipeline behaviors (such as logging, validation, metrics, and transaction management) are attached centrally to the mediator pipeline.
- **iOS Coordinator Pattern:** In iOS Swift development, the Coordinator pattern decouples UI ViewControllers from navigation logic. Instead of ViewController A instantiating and pushing ViewController B directly, ViewControllers notify a central `Coordinator`, which manages navigation transitions and deep linking.

- **GUI Dialog and Form Controllers:** Complex dialog windows (containing text boxes, checkboxes, sliders, and submit buttons) use a Form Mediator. When a user checks a box, the form mediator automatically enables, disables, or validates dependent fields without individual UI widgets referencing each other.
- **Real-Time Chat and Collaboration Gateways:** WebSocket servers function as mediators, managing room subscriptions, message formatting, and routing between active chat clients.

9 Observer and Interpreter Integration

In production-grade software architectures, the Mediator pattern frequently pairs with the Observer and Interpreter patterns:

9.1 Combining Mediator with the Observer Pattern (Event Aggregator)

While a standard Mediator pattern uses direct method invocations on colleague references, pairing it with the Observer pattern creates an **Event Aggregator / Event Bus**.

Colleague components register as observers with a central `EventMediator`. When an event occurs, the publishing colleague dispatches an event object to the Mediator without holding any reference to subscribers. The Mediator routes the payload to registered subscriber observers based on event types. This combines the unidirectional loose coupling of Observer with the centralized topology of Mediator.

9.2 Combining Mediator with the Interpreter Pattern

In dynamic, rule-driven systems—such as airport slot allocation engines, automated trade routing, or access control gateways—the Mediator's internal coordination rules should not be hardcoded into C++ source code.

Instead, the central `Mediator` embeds an **AST Interpreter**. When colleagues submit requests (e.g., flight landing requests with parameters such as fuel level, aircraft weight, and passenger count), the Mediator interprets dynamic domain-specific expressions (e.g., `"(fuel < 10%) OR (patient == CRITICAL)"`) at runtime to compute queue priority dynamically without requiring recompilation.

10 Conclusion

The Mediator Design Pattern provides an indispensable architectural solution for untangling complex, many-to-many component interactions. By replacing direct peer-to-peer dependencies with a centralized coordinator, the pattern eliminates $O(N^2)$ mesh complexity and establishes strict compliance with the Single Responsibility and Open-Closed Principles.

Through our C++17 implementation of the Airport Air Traffic Control simulation, we demonstrated how centralizing runway state machines, holding pattern queues, and emergency Mayday overrides in `ATCTower` allows specialized flight colleagues (`CommercialFlight`, `CargoFlight`, `EmergencyFlight`) to operate in total isolation from one another. When integrated with smart pointer lifetime management and combined with the Observer and Interpreter patterns, the Mediator pattern provides a robust, extensible foundation for real-world enterprise web backends, mobile UI coordinators, and real-time distributed systems.

References

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., & Stal, M. (1996). *Pattern-Oriented Software Architecture: A System of Patterns* (Vol. 1). John Wiley & Sons.
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Meyers, S. (2014). *Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14*. O'Reilly Media.
- Bogard, J. (2015). *MediatR: Simple, unambitious mediator implementation in .NET*. `https://github.com/jbogard/MediatR`
- Klishevich, S. (2015). *The Coordinator Pattern in iOS Architecture*. `https://khanlou.com/2015/01/the-coordinator/`
- Refactoring.Guru. (2024). *Design Patterns: Mediator Pattern*. `https://refactoring.guru/design-patterns/mediator`